

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина: Основы алгоритмизации и программирования

К защите допустить:
Ассистент кафедры информатики
_____ А. Д. Коростелев

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
на тему:

ИГРА В ЖАНРЕ ГОЛОВОЛОМКА

БГУИР КП 6–05 0612 02 007 ПЗ

Студент

И. Д. Ложечник

Руководитель

А. Д. Коростелев

Минск 2025

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»
Факультет компьютерных систем и сетей
Кафедра информатики

УТВЕРЖДАЮ
Заведующий кафедрой информатики
_____ Сиротко С.И.
« ____ » _____ 2025 г.

**ЗАДАНИЕ
по курсовой работе**

Группа 453505

Студенту Ложечнику Ивану Денисовичу

- 1. Тема работы:** Игра в жанре головоломка
- 2. Сроки сдачи студентом законченной работы:** 07.06.2025
- 3. Исходные данные к проекту:** Для написания курсового проекта был выбран фреймворк QT на основе языка программирования C++.
- 4. Содержание расчетно-пояснительной записки** (перечень подлежащих разработке вопросов):
Введение
Раздел 1. Обзор существующих аналогов
Раздел 2. Теоретическое обоснование разработки приложения
Раздел 3. Архитектура приложения
Раздел 4. Проектирование функций и алгоритмов, необходимых для работы приложения
Заключение.
Список использованной литературы.
Приложение 1. Листинг программного кода.
- 5. Консультанты по работе:** Коростелев А.Д.
- 6. Дата выдачи задания:** 27.02.2025 г
- 7. Календарный график работы над проектом на весь период проектирования** (с указанием сроков выполнения и трудоемкости отдельных этапов):

№ п/п	Наименование этапов курсового проекта	Срок выполнения этапов проекта	Примечание
1.	Поиск и написание теоретической информации	01.03.2025	10%
2.	Написание и отладка программы	01.04.2025	30%
3.	Демонстрация, заключение	01.05.2025	60%
4.	Защита курсовой работы	07.06.2025	Согласно графику

Руководитель _____ (Коростелев А.Д.)
Задание принял к исполнению 27.02.2025 _____
(_____)

СОДЕРЖАНИЕ

Введение.....	4
1 Обзор существующих аналогов	5
1.1 Краткий обзор игры Hitori.....	5
1.2 Краткий обзор правил и способов решения игры Hitori	6
1.3 Краткий обзор аналогов игры Hitori.....	8
1.4 Анализ недостающих функций и возможностей	10
2 Теоретическое обоснование разработки приложения	11
2.1 Обоснование необходимости разработки	11
2.2 Технологии программирования, используемые для решения поставленных задач.....	12
3 Архитектура приложения	14
3.1 Общая структура и архитектура приложения	14
3.2 Модуль аутентификации	15
3.3 Модуль главного меню	17
3.4 Модуль игрового окна	19
3.5 Модуль решения головоломки.....	22
3.6 Модуль хранения данных.....	23
3.7 Взаимодействие между модулями и масштабируемость	25
4 Проектирование функций и алгоритмов, необходимых для работы приложения	27
4.1 Общий анализ алгоритмов	27
4.2 Алгоритм решения заданного поля	27
4.3 Алгоритм генерации поля заданного размера	29
Заключение	31
Список использованной литературы	32
Приложение А (обязательное) Листинг программного кода.....	33

ВВЕДЕНИЕ

В настоящее время логические игры пользуются широкой популярностью среди людей разных возрастов. Они позволяют не только увлекательно провести время, но и способствуют развитию таких когнитивных способностей, как логическое мышление, внимание и память. Одной из таких игр является Hitori – японская головоломка, сочетающая в себе элементы логики и стратегии. Её основная задача заключается в том, чтобы, следуя определённым правилам, удалить часть ячеек из таблицы с числами, не нарушая условий уникальности и связности.

С переходом от бумажных носителей к цифровым, головоломки получили «вторую жизнь». Мобильные и настольные версии логических игр позволяют решать их в любом месте и в любое время, а также предлагают удобные функции, такие как автоматическое выделение ошибок, возможность отката действий и подсказки. Это делает головоломки более доступными и привлекательными для широкой аудитории.

Игра Hitori, как и другие головоломки японского происхождения, получила распространение во всём мире и стала популярной среди любителей интеллектуальных задач. Несмотря на то, что она менее известна, чем Судоку, её оригинальность и глубокий логический потенциал делают её достойным объектом для программной реализации.

Целью данного курсового проекта является разработка компьютерной версии игры Hitori.

Для достижения данной цели предусмотрены следующие задачи:

1 Изучение истории и правил классической игры для создания аналога Hitori.

2 Разработка версии аналога игры Hitori с использованием соответствующих инструментов (кроссплатформенный фреймворк Qt для языка C++). Тестирование алгоритмов игры на предмет возможных ошибок и неточностей.

3 Тестирование алгоритмов игры на предмет возможных ошибок и неточностей.

4 Анализ результата работы с целью определения возможных способов улучшения аналога.

Пояснительная записка оформлена в соответствии с СТП 01-2024 [1].

1 ОБЗОР СУЩЕСТВУЮЩИХ АНАЛОГОВ

1.1 Краткий обзор игры Hitori

Игра «Hitori» – это японская логическая головоломка, впервые опубликованная издательством Nikoli в марте 1990 года. Название «Hitori» (яп. ひとりにしてくれ) переводится как «Оставь меня в покое» или «один человек». Игра возникла как новая разработка среди множества логических головоломок, которые Nikoli публиковало в 1980–1990-х годах, включая такие известные игры, как судоку и шикаку.

После первой публикации в журнале *Puzzle Communication Nikoli* (см. рисунок 1.1) в 1990 году, игра быстро завоевала популярность среди любителей логических головоломок в Японии. Hitori отличалась от других головоломок того времени своей оригинальной механикой, объединяющей элементы исключения, поиска повторов и пространственного мышления. Это позволило ей занять достойное место среди других логических игр, таких как судоку, какуро и нонограммы.



Рисунок 1.1 – Современные журналы Nikoli

В течение 1990-х и 2000-х годов игра активно публиковалась в сборниках головоломок, как в Японии, так и за её пределами. С ростом интереса к японским логическим играм в Европе и Северной Америке, Hitori стала регулярно появляться в зарубежных изданиях, в том числе в специализированных журналах по головоломкам, а позже и в виде онлайн-версий на популярных игровых сайтах.

С развитием цифровых технологий игра получила новое дыхание. Появились мобильные приложения и компьютерные версии Hitori, предлагающие пользователю широкий выбор уровней сложности, автоматическую проверку правил и удобный интерфейс. Благодаря этим возможностям игра стала доступна широкой аудитории по всему миру, включая как новичков, так и опытных игроков, ценящих интеллектуальные вызовы.

На сегодняшний день Hitori занимает нишу среди популярных логических игр, оставаясь менее массовой, чем sudoku, но обладая стабильной базой поклонников. Она часто включается в подборки лучших японских головоломок, используется в образовательных целях для развития логического мышления и продолжает издаваться как в бумажном, так и в цифровом формате.

1.2 Краткий обзор правил и способов решения игры Hitori

Игра «Hitori» представляет собой логическую головоломку, в которой игроку предлагается квадратная таблица, заполненная числами. Основная задача состоит в том, чтобы путем затемнения определённых клеток добиться уникальности чисел в каждой строке и в каждом столбце, соблюдая при этом дополнительные ограничения. Правила игры: [2]

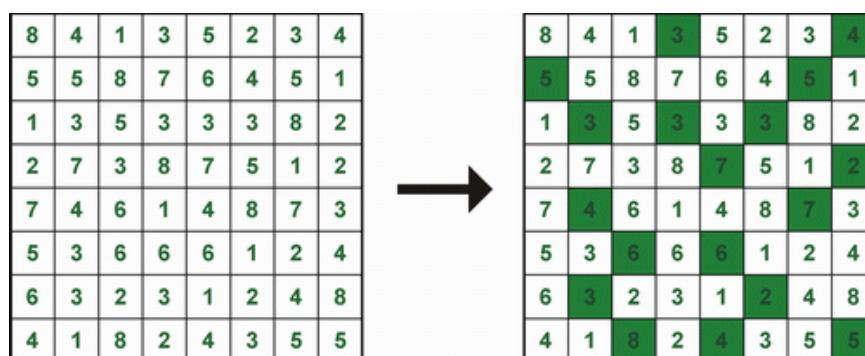
1 В каждой строке и в каждом столбце не должно быть повторяющихся чисел. Если одно и то же число встречается дважды или более, необходимо затемнить лишние вхождения так, чтобы осталась только одна копия этого числа в строке или столбце.

2 Затемнённые клетки не могут соприкасаться по горизонтали или вертикали. То есть, две или более затемнённых клетки не могут располагаться рядом. Однако они могут соприкасаться по диагонали.

3 Все незатемнённые клетки должны быть связаны между собой по горизонтали или вертикали, образуя единую непрерывную область. Это означает, что нельзя допускать изолированных групп незатемнённых клеток.

Решение головоломки требует внимательного анализа, логического мышления и стратегического подхода (см. рисунок 1.2). Игроку часто приходится делать предположения, проверять их на соответствие правилам и в

случае противоречий откатываться назад. Эти особенности делают игру не только интересной, но и полезной для развития когнитивных навыков.



8	4	1	3	5	2	3	4
5	5	8	7	6	4	5	1
1	3	5	3	3	3	8	2
2	7	3	8	7	5	1	2
7	4	6	1	4	8	7	3
5	3	6	6	6	1	2	4
6	3	2	3	1	2	4	8
4	1	8	2	4	3	5	5

8	4	1	3	5	2	3	4
5	5	8	7	6	4	5	1
1	3	5	3	3	3	8	2
2	7	3	8	7	5	1	2
7	4	6	1	4	8	7	3
5	3	6	6	6	1	2	4
6	3	2	3	1	2	4	8
4	1	8	2	4	3	5	5

Рисунок 1.2 – Поле Hitori и его решение

Следует подчеркнуть, что не каждое поле обладает решением, а тем более уникальным решением. Под уникальным подразумевается решение поля, имеющего единственное возможное решение, которое может быть получено исключительно на основе логических умозаключений без необходимости перебора вариантов. Такие поля представляют наибольший интерес для пользователей. Основные ходы для решения Hitori: [3]

- 1 Если клетка закрашена, то все соседние с ней не являются таковыми.
- 2 Если ячейка не закрашена, значит, все ячейки с тем же номером в соответствующих столбце и строке должны быть закрашенными.
- 3 Если ячейка отделяет белую часть сетки при закрашивании (то есть сделает ее несвязной), значит, ее закрашивать не надо.
- 4 Если три одинаковых числа идут подряд, центральное должно быть не закрашенным, другие – окрашены. Действительно, если центральное закрашено, то другие – нет, а потому будет два одинаковых числа в строке или столбце.
- 5 Если два одинаковых числа идут подряд, при том, что в той же строке/столбце есть еще одна ячейка с тем же номером, последняя должна быть закрашенной (иначе имеем две соседние закрашенные).
- 6 Если ячейка находится между двумя одинаковыми числами, она должна быть незакрашенной – иначе обе будут незакрашенными и стоять в одной строке/столбце.
- 7 Когда четыре одинаковых числа стоят в ячейках с одной общей вершиной, то две из них закрашены, а две – нет, причем есть два варианта (или верхний левый и нижний правый закрашены, или наоборот). Если квадрат, образуемый ими, находится в углу сетки, то есть только один вариант, потому что в другом нарушается правило о связности.

1.3 Краткий обзор аналогов игры Hitori

С момента появления игры «Hitori» в печатных изданиях, она обрела множество цифровых аналогов, доступных как в веб-версии, так и в виде мобильных приложений. Эти реализации позволяют пользователям решать головоломку в удобной интерактивной форме, часто предоставляя дополнительные функции, упрощающие игровой процесс.

Среди популярных онлайн-ресурсов можно выделить сайт <https://ru.puzzle-hitori.com> [4], где представлены разнообразные по сложности головоломки Hitori. Сайт предлагает понятный интерфейс, поддержку отмены ходов, а также подсказки. Однако стоит отметить, что визуальный стиль несколько устаревший (см. Рисунок 1.3), и отсутствует система прогресса или сохранения результата. Тем не менее, для игроков, желающих просто попрактиковаться в решении, этот ресурс вполне подходит.

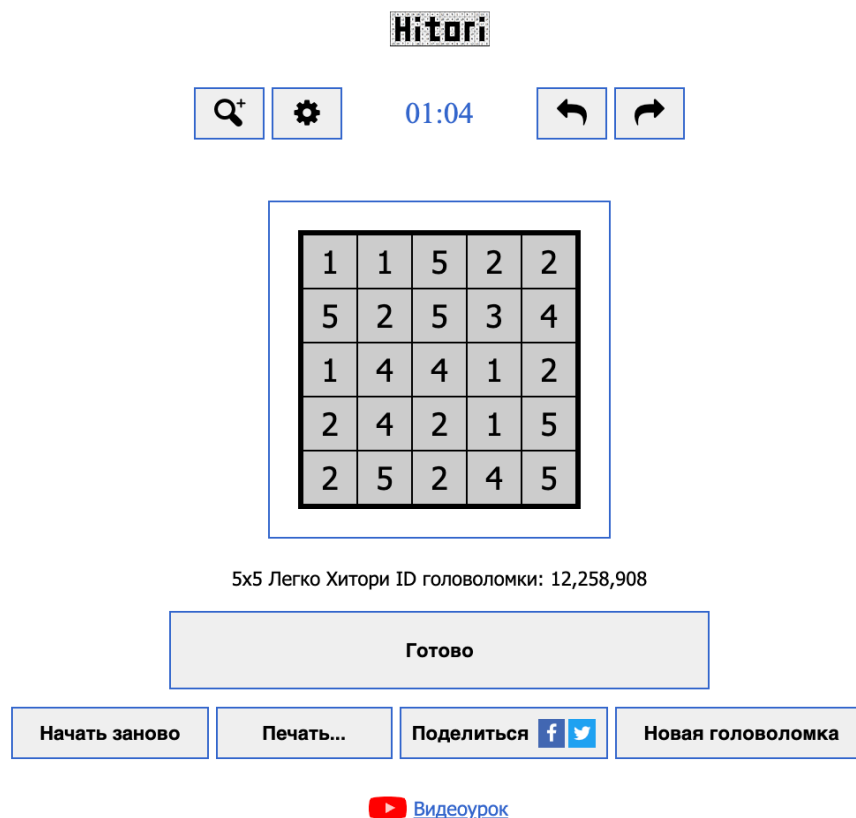


Рисунок 1.3 – Интерфейс игры Hitori [4]

Также интересным ресурсом является <https://www.conceptispuzzles.com>, который предоставляет большое количество логических игр, включая Hitori. Удобство сайта заключается в наличии личного кабинета, где можно отслеживать

прогресс, накапливать очки и участвовать в рейтингах. Кроме того, головоломки разбиты по уровню сложности, а интерфейс адаптирован для различных устройств. Среди недостатков можно отметить, что не весь контент доступен бесплатно – много задач требуют покупки. А также максимально простой интерфейс и отсутствие возможности посмотреть решение.

Мобильные приложения также активно развивают цифровую версию Hitori. Среди популярных можно выделить:

1 "Hitori" от разработчика Christian Lueth (доступна в Google Play и App Store) – приложение отличается графикой, наличием обучающего режима и функцией автоматической проверки правил (см. рисунок 1.4). Уровни варьируются от простых до очень сложных, что делает игру подходящей как для новичков, так и для опытных пользователей. Однако в бесплатном доступе находятся только около 50 полей Hitori.

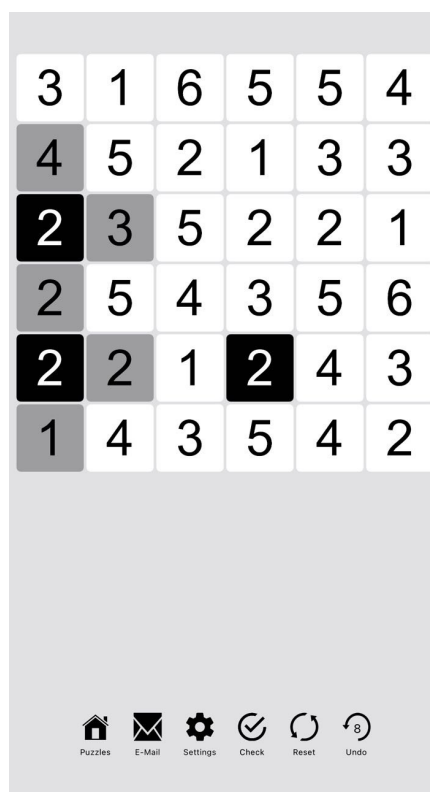


Рисунок 1.4 – Интерфейс мобильного приложения Hitori

2 "Hitori Puzzle" от Branium Studios – предлагает минималистичный дизайн и быстрое управление. Приложение не перегружено лишними функциями, что особенно удобно для коротких игровых сессий. Из минусов – ограниченные возможности настройки интерфейса.

3 "Conceptis Hitori" – официальное мобильное приложение от Conceptis

Ltd., которое повторяет функциональность сайта. Преимущество в удобном сенсорном управлении и доступе к новым задачам каждую неделю. Внутри приложения реализована система покупок, только базовые уровни доступны бесплатно.

1.4 Анализ недостающих функций и возможностей

В результате анализа приложений были определены следующие необходимые требования к приложению:

1 Корректная генерация поля. Пользователь должен иметь выбор: решать любое поле или только уникальное.

2 В свободном доступе должно быть большое количество полей разных размеров.

3 Наличие заметок.

4 Удобное заполнение поля.

5 Наличие таймера.

6 Индикатор текущего решения чтобы пользователь сразу после окрашивания клетки понимал возможно ли решить головоломку при текущей раскраске поля.

7 Сохранение статистики игры.

2 ТЕОРЕТИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ ПРИЛОЖЕНИЯ

2.1 Обоснование необходимости разработки

На сегодняшний день игровая индустрия является одной из самых стремительно развивающихся, прибыльных и востребованных сфер в мире. Она не только охватывает многомиллионную аудиторию игроков всех возрастов и интересов, но и активно влияет на другие сферы культуры, образования и бизнеса. Каждый день появляются десятки новых видеоигр, настольных игр и мобильных приложений, предлагающих как совершенно оригинальные концепции, так и усовершенствованные версии уже известных проектов. В их числе – аналоги классических игр, таких как Hitori, которые позволяют не просто испытать знакомые эмоции, но и по-новому взглянуть на любимую механику.

Появление и распространение аналогов популярных игр, таких как Hitori, свидетельствует о востребованности и устойчивом интересе к оригинальной идее. Однако важно понимать, что аналог – это не простая копия, а, как правило, творческая переработка концепции с добавлением новых механик, улучшением дизайна, устранением недостатков оригинала и расширением возможностей игры. Разработчики таких проектов не просто повторяют чужие идеи – они глубоко погружаются в суть оригинальной игры, изучают её сильные и слабые стороны, перенимают удачные находки и стараются привнести что-то уникальное, основанное на личном опыте и видении.

Разработка собственной версии Hitori – это захватывающее и многогранное приключение, включающее в себя множество этапов, каждый из которых требует определённых знаний, умений и творческого подхода. Процесс начинается с поиска и проработки идеи. На этом этапе необходимо определить уникальные черты будущей игры – её «изюминку»: чем она будет отличаться от других версий, какие механики будут использоваться, какое настроение и стиль будет преобладать, и какой сюжет (если он предполагается) станет основой игрового процесса. Затем следует выбор инструментов и технологий для реализации проекта – это может быть язык программирования, графический движок, библиотеки пользовательского интерфейса и другие средства.

Далее следует создание прототипа, который позволяет оценить основные элементы игры на практике и выявить потенциальные проблемы. После этого начинается основной этап разработки, включающий программирование, разработку интерфейса, отрисовку графики и реализацию логики. Немаловажным этапом является тестирование, в ходе которого выявляются ошибки, проводится доработка и шлифовка всех компонентов игры. И, наконец, осуществляется запуск готового продукта, после чего возможна его дальнейшая поддержка и развитие на основе отзывов пользователей.

Разработка аналога Hitori предоставляет целый ряд значимых преимуществ:

1 Творческая переработка существующей идеи – возможность внести оригинальные дополнения в классическую механику, комбинируя сильные стороны уже существующих версий и предлагая новые функции.

2 Приобретение опыта в сфере геймдева – непосредственное участие в процессе разработки способствует лучшему пониманию структуры игровых проектов, что особенно ценно для начинающих разработчиков.

3 Развитие технических и художественных навыков – реализация проекта позволяет значительно улучшить навыки программирования, дизайна интерфейсов, работы с графикой, а также управления проектами.

2.2 Технологии программирования, используемые для решения поставленных задач

Выбор языка программирования – важное решение, определяющее успех всего проекта. Для разработки аналога Hitori был выбран язык C++.

Выбор языка C++ обосновывается его высокой производительностью и популярностью при создании игр. Язык C++ – мощный инструмент разработки приложений. Язык обеспечивает удобство и легкость написания кода, что является важным фактором при разработке сложных программных продуктов. Были рассмотрены его основные свойства:

1 Мультипарадигмальный язык программирования. C++ сочетает в себе свойства как высокоуровневых (например, поддержка объектно-ориентированного программирования), так и низкоуровневых языков программирования (например, доступ к памяти, указателям и битам).

2 Компилируемый язык программирования. Это означает, что перед запуском код, написанный на языке C++, необходимо преобразовать в машинный код. В результате компиляции будет создан исполняемый файл, запускаемый на компьютере и выполняемый операционной системой.

3 Статически типизированный язык. Статическая типизация подразумевает, что типы данных определяются на этапе компиляции и не могут быть изменены во время выполнения.

4 Не строго типизированный язык. Язык C++ допускает возможность преобразования типов.

Для создания удобного графического интерфейса взаимодействия с пользователем был выбран Qt – кроссплатформенный фреймворк, предназначенный для создания графических интерфейсов предложений. Использование данного фреймворка облегчает процесс разработки, предоставляя большое количество инструментов и библиотек. Широкий функционал позволяет

уделить больше внимания разработке логики приложения, а не созданию графического интерфейса.

Полезной характеристикой Qt является его модульность. Фреймворк состоит из множества модулей, который обеспечивает удобные инструменты для написания, компиляции, запуска кода, тестирования и отладки.

При создании приложения важно выбрать удобную среду разработки. В рамках данного курсового проекта были выбраны CLion – среда разработки от JetBrains, которая предоставляет множество функций, способствующих эффективной разработке программного обеспечения и QT Creator - кроссплатформенная свободная IDE для разработки на C, C++ и QML от Trolltech для работы с фреймворком QT.

3 АРХИТЕКТУРА ПРИЛОЖЕНИЯ

3.1 Общая структура и архитектура приложения

Приложение, разработанное в рамках курсовой работы, представляет собой комплексное программное обеспечение, реализованное с использованием кроссплатформенной библиотеки Qt, что обеспечивает его совместимость с различными операционными системами, включая Windows, macOS и Linux. Архитектура приложения спроектирована с учетом принципов модульности, расширяемости и поддерживаемости, что позволяет легко вносить изменения в функциональность, добавлять новые возможности или адаптировать проект под различные требования. Структура приложения включает в себя несколько ключевых компонентов, таких как интерфейс пользователя, модули управления игровым процессом, системы аутентификации и хранения данных, а также вспомогательные утилиты для обработки игровых данных. Рассмотрим архитектуру и структуру приложения более подробно, уделяя внимание каждому аспекту и описывая взаимосвязь между компонентами.

Приложение следует архитектурному шаблону Model-View-Controller (MVC), адаптированному под особенности разработки графических интерфейсов с использованием Qt. В данном случае модель (Model) представляет собой данные игры, включая игровое поле, правила Hitori и статистику пользователей; представление (View) реализовано через графический интерфейс, созданный с помощью Qt Designer и управляемый классами Qt Widgets; а контроллер (Controller) отвечает за логику взаимодействия между моделью и представлением, включая обработку пользовательского ввода и обновление состояния игры. Такой подход позволяет разделить бизнес-логику от пользовательского интерфейса, что упрощает тестирование и модификацию отдельных компонентов без влияния на остальные части системы.

На верхнем уровне архитектура приложения включает следующие основные модули:

1 Модуль аутентификации – отвечает за управление входом и регистрацией новых пользователей.

2 Модуль главного меню – предоставляет пользователю доступ к основным функциям приложения.

3 Модуль игрового окна – реализует основной игровой процесс, включая отображение игрового поля и обработку действий пользователя.

4 Модуль решения головоломки – содержит множество алгоритмов для автоматического решения головоломки Hitori.

5 Модуль хранения данных – обеспечивает сохранение и загрузку данных пользователей и игровых досок.

На рисунке 3.1 изображена структура проекта в среде разработки QT Creator.

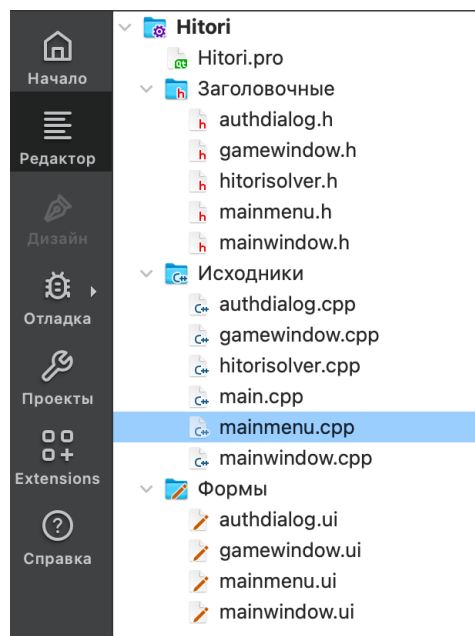


Рисунок 3.1 – Структура проекта Hitori

3.2 Модуль аутентификации

Модуль аутентификации является одной из ключевых составляющих приложения, обеспечивая персонализацию пользовательского опыта и управление доступом к игровым функциям. Этот модуль реализован через класс AuthDialog, который представляет собой диалоговое окно, предоставляющее пользователю возможность либо войти в систему под существующей учетной записью, либо зарегистрировать новую. Основная цель модуля заключается в том, чтобы гарантировать, что каждый пользователь имеет уникальный профиль, связанный с его статистикой, достижениями и игровым прогрессом. Модуль аутентификации также отвечает за базовую защиту данных пользователя, хотя текущая реализация имеет определенные ограничения, которые будут рассмотрены ниже. Рассмотрим функциональные возможности, архитектуру, особенности реализации, а также преимущества и недостатки данного модуля, уделяя внимание его интеграции с другими частями приложения.

Модуль аутентификации предоставляет пользователю две основные функции: вход в систему и регистрацию. При запуске приложения пользователю

отображается диалоговое окно, содержащее два поля ввода: одно для логина и второе для пароля. Поле пароля настроено в режиме скрытого ввода, что обеспечивает минимальный уровень конфиденциальности при вводе данных. Для выполнения входа или регистрации пользователь может воспользоваться соответствующими кнопками: "Войти" и "Зарегистрироваться".

Процесс входа предполагает, что пользователь уже зарегистрирован, и его данные хранятся в системе. Модуль проверяет введенные логин и пароль на соответствие записям, сохраненным в текстовом файле `users.txt`, который используется в качестве простейшей базы данных. Если совпадение найдено, диалоговое окно закрывается с результатом успешного выполнения, что позволяет приложению продолжить работу, передав имя пользователя в основной поток программы. В случае, если логин или пароль неверны, пользователю отображается сообщение об ошибке с предложением повторить попытку или зарегистрироваться.

Регистрация, в свою очередь, позволяет создать нового пользователя. Модуль сначала проверяет, не существует ли уже пользователь с таким логином, чтобы избежать дублирования учетных записей. Если логин уникален, данные пользователя (логин и пароль) записываются в файл `users.txt`, а также создается отдельный файл статистики для данного пользователя, который будет использоваться для хранения информации о его игровых достижениях. После успешной регистрации пользователю отображается сообщение об успехе, и он может сразу войти в систему, используя введенные данные.

Теперь рассмотрим реализацию модуля аутентификации. Класс `AuthDialog` наследуется от базового класса `QDialog`, что делает его модальным диалоговым окном, блокирующим взаимодействие с основным окном приложения до завершения процесса аутентификации. Интерфейс диалога формируется с использованием `Qt Designer`, а сгенерированный файл `ui_authdialog.h` содержит описание всех графических элементов, таких как поля ввода (`loginEdit` и `passwordEdit`) и кнопки (`loginButton` и `registerButton`). Эти элементы связаны с функциональностью через механизм сигналов и слотов `Qt`, который позволяет реагировать на действия пользователя, такие как нажатие кнопок.

Основная логика работы модуля реализована в трех ключевых методах: проверка существования пользователя, сохранение нового пользователя и обработка событий входа и регистрации. Метод проверки существования пользователя открывает файл `users.txt` в режиме чтения и построчно проверяет наличие записи с указанным логином. Если запись найдена, метод возвращает подтверждение, что пользователь уже существует. Этот метод хорош, но не подходит для больших объемов данных, так как приходится последовательно

считывать весь файл, что занимает слишком много времени.

Сохранение нового пользователя осуществляется через метод, который открывает users.txt в режиме добавления и записывает новую строку в формате "логин:пароль". После этого создается файл статистики для пользователя в директории statistics/, где записываются начальные значения (например, нули для количества игр, побед и других метрик). Этот файл будет использоваться модулем хранения данных для отслеживания прогресса пользователя.

Визуальный стиль диалога аутентификации играет важную роль в создании первого впечатления у пользователя. Фоновое изображение, применяемое через QPalette, добавляет индивидуальности интерфейсу, делая его более привлекательным по сравнению с типичным серым диалогом Qt. Интерфейс окна аутентификации можно увидеть на рисунке 3.2.

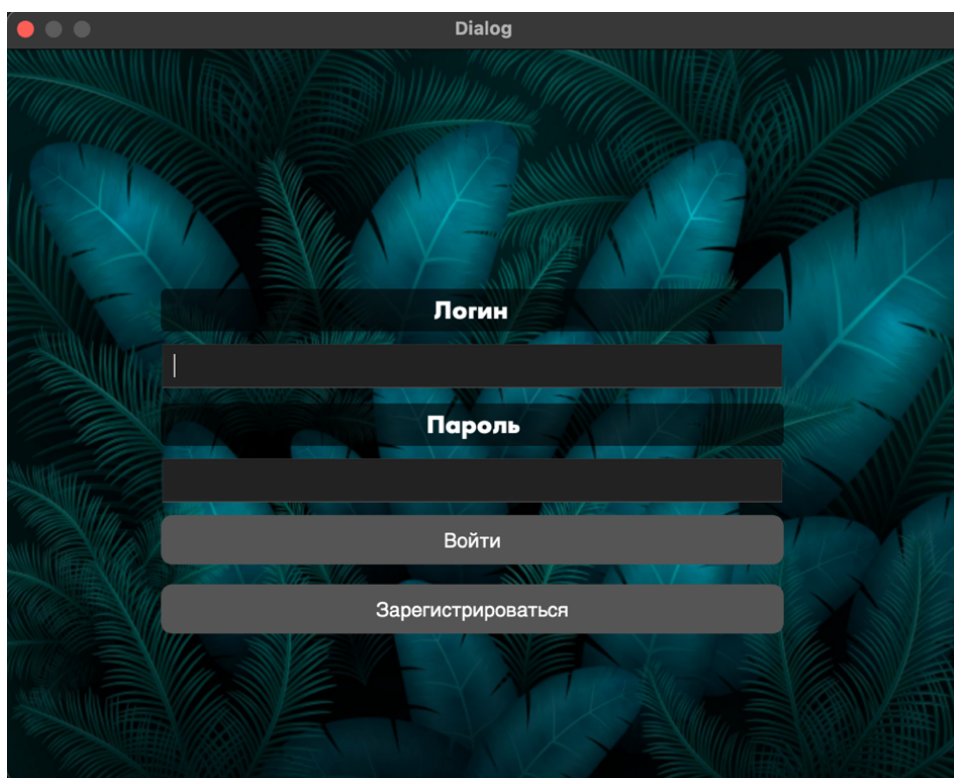


Рисунок 3.2 – Интерфейс окна аутентификации

3.3 Модуль главного меню

Модуль главного меню, реализованный через класс MainMenu, представляет собой центральный узел управления приложением, обеспечивая пользователю интуитивно понятный доступ к основным функциям после успешной аутентификации. Этот модуль, наследующийся от QMainWindow –

базового класса Qt для создания окон с расширенными возможностями, включая панель меню, статусную строку и центральный виджет, играет ключевую роль в организации пользовательского опыта. Модуль MainMenu включает графический интерфейс, разработанный с использованием Qt Designer, что делает его функциональным элементом приложения. Рассмотрим архитектуру, функциональные возможности, визуальные аспекты, интеграцию с другими модулями, а также преимущества, недостатки и потенциальные пути улучшения данного модуля, уделяя внимание деталям его реализации и значимости для общей структуры приложения.

Модуль главного меню, реализованный через класс MainMenu, предоставляет пользователю точку входа в игровую среду после завершения процесса аутентификации в AuthDialog. После успешного входа или регистрации имя пользователя передается в MainMenu, где оно может быть использовано для персонализации интерфейса, например, отображения приветственного сообщения или загрузки индивидуальной статистики. Основная функция модуля заключается в запуске игры через переход к модулю игрового окна (GameWindow), что осуществляется с возможностью выбора параметров, таких как размер игрового поля (5x5, 10x10, 15x15, 20x20). Этот выбор может быть реализован через выпадающий список или комбинированный виджет, что позволяет пользователю адаптировать игровой процесс под свои предпочтения.

Помимо запуска игры, модуль MainMenu потенциально включает дополнительные функции, такие как просмотр статистики пользователя, доступ к правилам игры или выход из приложения. Интерфейс, созданный в Qt Designer, содержит элементы управления, такие как кнопки и метки, которые могут быть связаны с соответствующими действиями. Например, кнопка "Начать игру" может запускать GameWindow с выбранным размером доски, а кнопка "Статистика" – отображать данные из файла статистики пользователя, хранящегося в директории statistics/.

Класс MainMenu построен на основе QMainWindow, что обеспечивает гибкость для добавления панели меню, статусной строки или дополнительных виджетов. Интерфейс модуля определяется файлом ui_mainmenu.h, сгенерированным из макета, созданного в Qt Designer. Этот файл описывает расположение и свойства графических элементов, таких как кнопки, метки и, возможно, выпадающие списки, которые составляют пользовательский интерфейс. Конструктор MainMenu инициализирует интерфейс с помощью ui->setupUi(this) и может включать логику для обработки переданного имени пользователя, например, его отображение в заголовке окна или использование для загрузки данных.

Архитектура модуля следует принципам модульности и объектно-ориентированного программирования. Логика взаимодействия с пользователем реализована через механизм сигналов и слотов Qt, где кнопки и другие элементы интерфейса связаны с соответствующими слотами. Например, слот, связанный с кнопкой "Начать игру", может создавать экземпляр GameWindow с передачей параметров, таких как имя пользователя и размер доски, выбранный из QComboBox. Эта структура позволяет легко расширять функциональность, добавляя новые слоты для обработки дополнительных действий, таких как просмотр статистики или выход.

Визуальный стиль модуля главного меню определяется макетом, созданным в Qt Designer, и может быть дополнительно настроен через QStyleSheet для обеспечения единого дизайна с другими частями приложения. Интерфейс окна главного меню можно увидеть на рисунке 3.3.

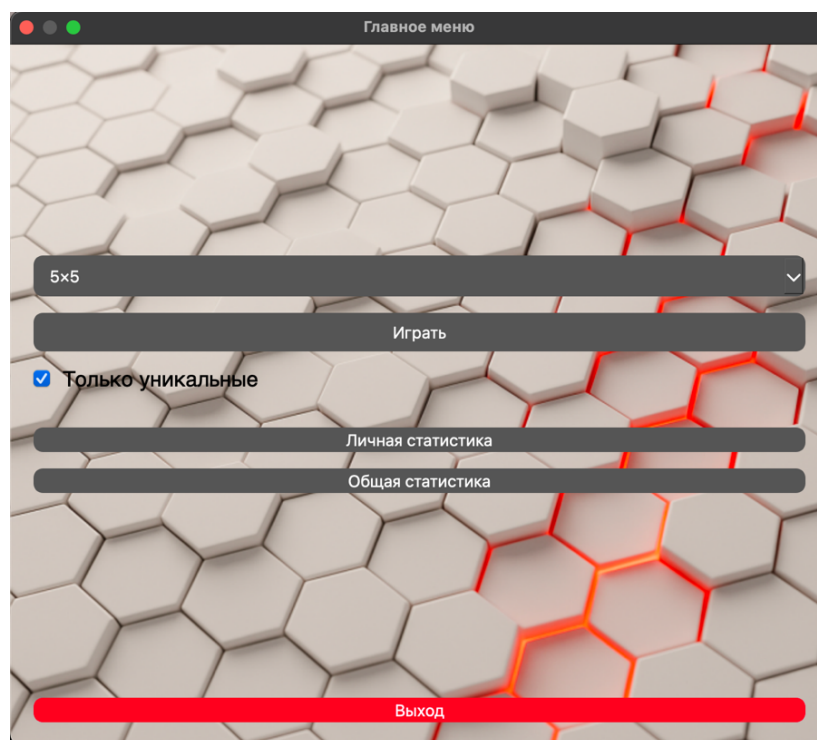


Рисунок 3.3 – Интерфейс окна главного меню

3.4 Модуль игрового окна

Модуль игрового окна, реализованный через класс GameWindow, представляет собой ядро приложения, обеспечивая непосредственное взаимодействие пользователя с игровой логикой и графическим интерфейсом головоломки. Этот модуль, наследующийся от QMainWindow – базового класса

Qt для создания окон с расширенными возможностями, является центральным компонентом, где реализуется основной игровой процесс, включая отображение игрового поля, обработку действий пользователя и управление временными параметрами. Интерфейс модуля спроектирован с использованием Qt Designer и описан в файле `gamewindow.ui`, что обеспечивает гибкость и визуальную согласованность с другими частями приложения. Рассмотрим архитектуру, функциональные возможности, визуальные аспекты, интеграцию с другими модулями, а также преимущества, недостатки и потенциальные пути улучшения данного модуля, уделяя внимание деталям его реализации и значимости для общей структуры приложения.

Модуль игрового окна предоставляет пользователю полный набор инструментов для игры в Hitori, включая загрузку и отображение игрового поля, изменение состояния ячеек (белый, серый, черный), отслеживание времени и проверку правильности решения. Класс `GameWindow` принимает три параметра при создании: имя пользователя (`username`), размер доски (`size`) и флаг использования уникальных досок (`useUnique`), что позволяет персонализировать игровой процесс и варьировать сложность. Игровое поле генерируется динамически на основе данных, загруженных из файлов (например, `10x10.txt` или `10x10_unique.txt`), с использованием случайного выбора одной из доступных конфигураций.

Основные функции модуля включают:

- 1 Инициализация игрового поля – формирование таблицы кнопок, представляющих ячейки доски, с числами, загруженными из файла.
- 2 Обработка кликов – изменение состояния ячеек (белый → серый → черный → белый) в ответ на действия пользователя, что отражает правила Hitori.
- 3 Отслеживание времени – использование таймера для подсчета времени, прошедшего с начала игры, с отображением в формате "минуты:секунды".
- 4 Проверка решения – исследование текущего состояния доски с использованием модуля решения (`HitoriSolver`) и индикация правильности через цветной индикатор.
- 5 Сдача и показ решения – остановка игры по запросу пользователя с отображением правильного решения, полученного от `HitoriSolver`.
- 6 Обновление статистики – сохранение данных о завершении игры (общее количество побед, побед по размеру доски, лучшее время) в файл статистики пользователя.

Дополнительно модуль обеспечивает возврат к главному меню (`MainMenu`) после завершения игры или сдачи, передавая имя пользователя для сохранения контекста. Индикатор правильности решения, реализованный через виджет с

изменяемым фоном (зеленый для верного, красный для неверного), добавляет интерактивности и визуальной обратной связи.

Класс `GameWindow` построен на основе `QMainWindow`, что позволяет интегрировать панель меню, статусную строку и центральный виджет, описанный в `gamewindow.ui`. Интерфейс включает ключевые элементы: `boardWidget` (контейнер для игрового поля), `surrenderButton` (кнопка сдачи), `timeLabel` (отображение времени) и `indicator` (индикатор правильности). Конструктор класса инициализирует интерфейс с помощью `ui->setupUi(this)` и выполняет ряд операций: проверку допустимого размера доски, создание таймера, загрузку данных, инициализацию интерфейса и центрирование окна.

Визуальный стиль модуля игрового окна определяется макетом из `gamewindow.ui` и кастомными стилями, применяемыми через `QStyleSheet`. Игровое поле представлено сеткой кнопок, с белым фоном по умолчанию, черной границей и черным текстом, отображающим числа. При изменении состояния ячеек стиль обновляется: серый цвет (`#aaaaaa`) используется для промежуточного состояния, черный с белым текстом – для финального выбора. Шрифт настроен на 16 пикселей с жирным начертанием для четкости, а отступы и поля минимизированы для компактности.

Полный интерфейс игрового окна можно увидеть на рисунке 3.4.

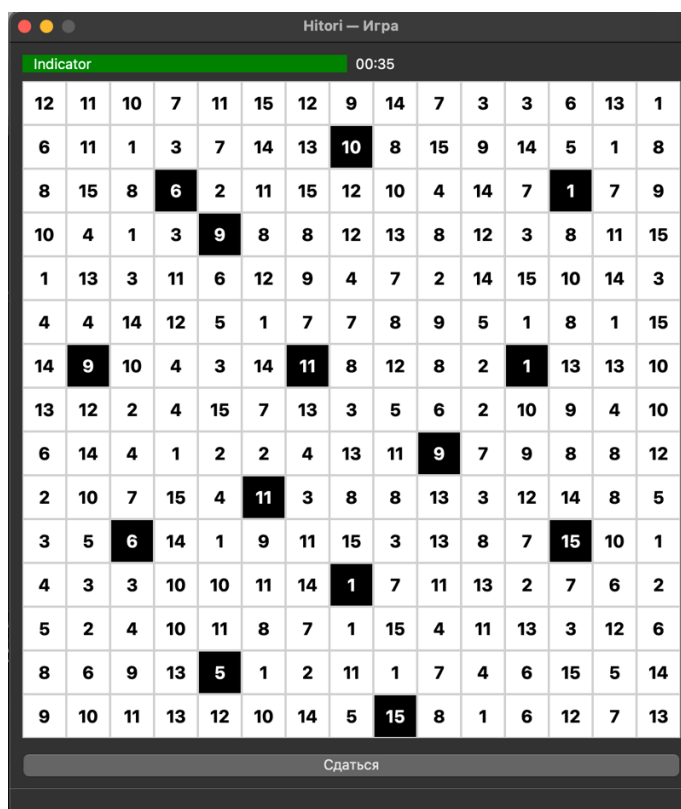


Рисунок 3.4 – Интерфейс игрового окна

3.5 Модуль решения головоломки

Модуль решения головоломки, реализованный через класс `HitoriSolver`, является важным компонентом приложения, обеспечивая автоматическое нахождение решения головоломки `Hitori`, проверку решений пользователя и предоставление ответа. Этот модуль играет ключевую роль в повышении интерактивности игры, позволяя пользователю сравнивать свои действия с правильным решением или получать помощь при затруднениях. Класс `HitoriSolver` представляет собой независимую библиотеку логики, которая использует рекурсивные алгоритмы и эвристические подходы для анализа игрового поля и нахождения оптимального решения в соответствии с правилами `Hitori`. Рассмотрим его архитектуру, функциональные возможности, алгоритмическую основу и интеграцию с другими модулями.

Модуль решения головоломки предоставляет три основные функции, которые интегрируются с другими частями приложения для обеспечения непрерывного игрового процесса:

- 1 Нахождение решения (`GetHitoriSolve`) – принимает размер поля и начальную конфигурацию доски, возвращая пару из булевого значения (успех/неуспех) и двумерного вектора, представляющего решение, где 1 обозначает закрашенные ячейки, а 0 – незакрашенные.

- 2 Проверка решения (`CheckSolution`) – анализирует текущее состояние доски, переданное в виде двумерного вектора, и возвращает булево значение, указывающее, соответствует ли решение правилам `Hitori`.

- 3 Вспомогательные проверки – включает ряд методов для анализа и обработки поля, таких как проверка связности, обработка дублирующихся чисел и устранение противоречий.

Эти функции позволяют модулю быть универсальным инструментом, который используется в `GameWindow` для отображения правильного решения при сдаче, проверки текущего состояния доски в процессе игры и предоставления визуальной обратной связи через индикатор правильности. Модуль также поддерживает масштабируемость, позволяя работать с досками разного размера (5x5, 10x10, 15x15, 20x20), что делает его гибким для различных уровней сложности.

Класс `HitoriSolver` представляет собой статическую библиотеку логики, где все методы объявлены как `static`, что позволяет вызывать их без создания экземпляра класса. Это архитектурное решение упрощает использование модуля, так как он не требует управления состоянием объекта, но может ограничивать масштабируемость в случае необходимости добавления динамических данных.

Основные данные хранятся в статическом поле `GeneralAnswer`, которое используется для передачи найденного решения между методами и возврата результата вызывающей стороне.

Алгоритмическая основа модуля включает несколько этапов:

1 Инициализация поля: Метод `GetHitoriSolve` начинается с создания начального поля решений (`curAnswer`), где каждая ячейка изначально помечена как "неопределенная" (2). Затем ячейки, которые не имеют дубликатов в строке или столбце, помечаются как незакрашенные (0), так как по правилам `Hitori` такие ячейки не могут быть закрашены.

2 Эвристическая обработка (`FieldProcessing`): Этот метод итеративно применяет ряд проверок для устранения противоречий.

3 Рекурсивный поиск (`SolveHitori`): Если после эвристической обработки остаются неопределенные ячейки, метод рекурсивно пробует закрасить каждую такую ячейку, проверяя, приводит ли это к допустимому решению. Первое найденное решение сохраняется в `GeneralAnswer`.

4 Проверка решения (`CheckSolution`): Анализирует переданное решение, проверяя отсутствие соседних закрашенных ячеек и связность незакрашенных через `DisconnectedProcessing`.

Метод `CountBFS` использует обход в ширину для подсчета связных незакрашенных ячеек, что является ключевым элементом проверки связности поля. Алгоритм `BFS` реализован с использованием двух очередей (`X` и `Y`) для хранения координат, что позволяет эффективно обходить поле, избегая закрашенных ячеек.

Более подробно требуемые для проекта алгоритмы будут рассмотрены в главе 4.

3.6 Модуль хранения данных

Модуль хранения данных в приложении отвечает за сохранение и загрузку информации о пользователях, их игровых достижениях, а также данных игровых полей, обеспечивая персонализацию и доступ к головоломкам. Этот модуль реализован через использование локальных текстовых файлов, что упрощает реализацию, но ограничивает масштабируемость. Основные компоненты включают файл `users.txt` для учетных данных, индивидуальные файлы статистики в директории `statistics/` (например, `[username].txt`), и файлы игровых полей в директории `boards/`, такие как `5x5.txt` и `5x5_unique.txt`. Рассмотрим архитектуру, функциональные возможности, интеграцию, а также преимущества и недостатки модуля.

Модуль выполняет три основные функции: управление учетными данными, хранение статистики и предоставление игровых полей. Файл `users.txt` содержит пары "логин:пароль" для проверки доступа в модуле аутентификации. Файлы статистики в `statistics/` хранят девять строк: общее количество побед, а также победы и лучшее время для размеров 5x5, 10x10, 15x15, 20x20. Директория `boards/` содержит текстовые файлы с конфигурациями головоломок и их количества (например, `10x10.txt`, `10x10_unique.txt`, `10x10_cnt.txt`), которые загружаются в `GameWindow` для генерации поля. Обновление статистики происходит в `GameWindow::updateStatistics` после победы, а загрузка полей – в `GameWindow::loadBoard`.

Модуль использует `QFile` и `QTextStream` для работы с файлами. Данные записываются и читаются построчно, что обеспечивает простоту, но требует последовательного доступа. Путь к файлам жестко задан (например, `/Users/senior/.../statistics/` и `/Users/senior/.../boards/`), что связывает модуль с другими компонентами. Метод `saveUser` в `AuthDialog` добавляет записи в `users.txt` и создает файл статистики, `updateStatistics` обновляет данные, а `loadBoard` читает файлы из `boards/`.

Модуль интегрирован с `AuthDialog` для сохранения учетных данных, с `GameWindow` для обновления статистики и загрузки полей из `boards/`, и косвенно с `MainMenu` через имя пользователя. Это обеспечивает персонализацию и доступ к данным.

Содержимое папки `boards` можно увидеть на рисунке 3.5.

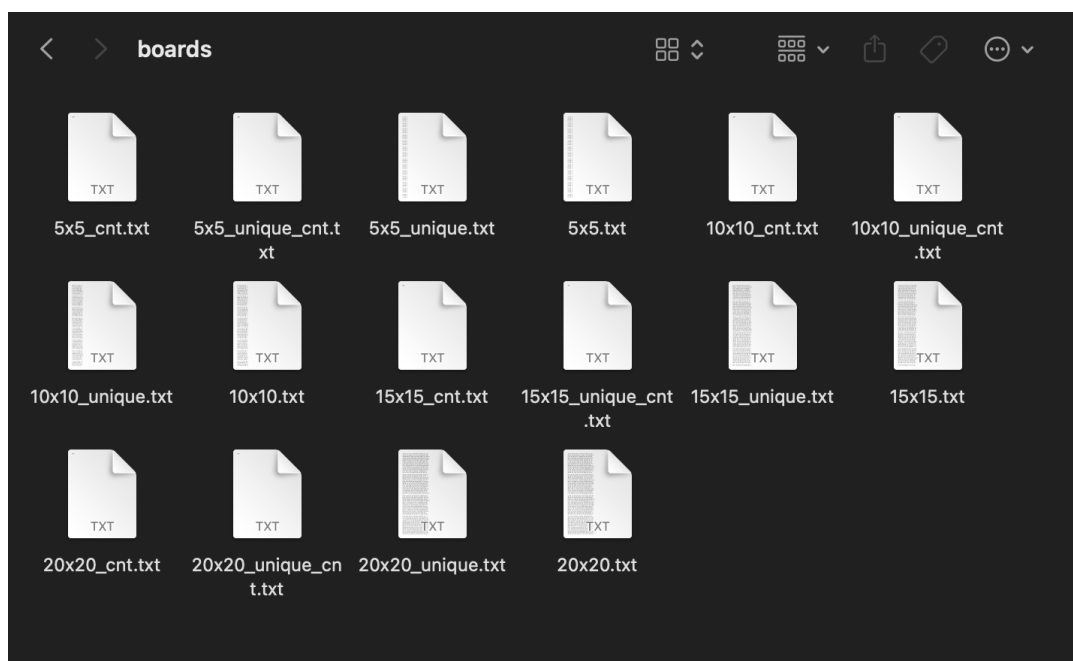


Рисунок 3.5 – Хранение полей Hitori

3.7 Взаимодействие между модулями и масштабируемость

Модуль взаимодействия между компонентами приложения и его масштабируемость являются ключевыми аспектами архитектуры, которые обеспечивают целостность системы и её способность к дальнейшему развитию. Взаимодействие между модулями построено на принципах модульности и асинхронности, что позволяет эффективно управлять потоками данных и событиями, в то время как масштабируемость системы определяет её потенциал для расширения функциональности и адаптации к новым требованиям. Рассмотрим, как модули приложения взаимодействуют друг с другом, какие механизмы используются для передачи данных, а также какие шаги можно предпринять для улучшения масштабируемости и устойчивости системы.

Взаимодействие между модулями в приложении реализовано через механизм сигналов и слотов Qt, который обеспечивает асинхронное и гибкое управление событиями. Этот подход позволяет модулям оставаться независимыми, но при этом эффективно обмениваться данными и реагировать на действия пользователя. Например, модуль аутентификации (AuthDialog) передает имя пользователя в модуль главного меню (MainMenu) после успешного входа или регистрации. Это происходит через вызов метода getUsername(), после чего AuthDialog завершает свою работу с результатом QDialog::Accepted, сигнализируя MainMenu о необходимости продолжить выполнение. Такой подход позволяет четко разделить ответственность: AuthDialog отвечает только за аутентификацию, а MainMenu использует полученные данные для дальнейшей инициализации.

Модуль главного меню (MainMenu) играет роль центрального узла, который связывает пользователя с игровым процессом. После получения имени пользователя он создает экземпляр GameWindow с передачей параметров, таких как размер доски (например, 5x5 или 10x10) и флаг использования уникальных досок. Эти параметры могут быть выбраны через элементы интерфейса, такие как выпадающий список, и передаются через конструктор GameWindow. В свою очередь, GameWindow взаимодействует с модулем хранения данных для загрузки игрового поля из директории boards/ (например, 10x10.txt) и обновления статистики в директории statistics/ после завершения игры. Этот процесс включает вызов метода updateStatistics, который записывает данные о победах и времени в файл [username].txt, обеспечивая персонализацию.

Модуль игрового окна (GameWindow) также активно взаимодействует с модулем решения головоломки (HitoriSolver). Во время игры метод checkSolution вызывает HitoriSolver::CheckSolution для проверки текущего состояния доски,

обновляя индикатор правильности. При сдаче вызывается `HitoriSolver::GetHitoriSolve`, который возвращает правильное решение, отображаемое на поле. После завершения игры или сдачи `GameWindow` создает новый экземпляр `MainMenu`, передавая имя пользователя, что позволяет пользователю вернуться в меню и начать новую игру. Этот циклический процесс демонстрирует, как модули связаны через передачу данных и события (клик по кнопке, завершение игры).

Модуль хранения данных, включающий файлы `users.txt`, директорию `statistics/` и `boards/`, выступает в качестве пассивного компонента, предоставляя данные другим модулям. Например, `AuthDialog` читает и записывает учетные данные в `users.txt`, `GameWindow` загружает поля из `boards/` и обновляет статистику, а потенциально `MainMenu` может использовать данные из `statistics/` для отображения рейтинга. Связь между модулями осуществляется через имя пользователя, которое служит ключом для доступа к персонализированным данным, что подчеркивает важность модульности и инкапсуляции.

Одним из преимуществ масштабируемости приложения является модульная структура, которая позволяет добавлять новые функции с минимальными изменениями. Например, для поддержки новых размеров досок (например, 25x25) достаточно добавить соответствующие файлы в директорию `boards/` и обновить логику в `GameWindow` для их обработки. Аналогично, добавление новых типов головоломок (например, с другими правилами) потребует лишь расширения `HitoriSolver` и добавления новых файлов в `boards/`.

В заключении отмечу, что взаимодействие между модулями в приложении реализовано эффективно благодаря сигналам и слотам Qt, обеспечивая асинхронность и модульность. Однако для большей эффективности можно использовать базы данных, `QSettings` и параллелизацию [5], [6], это позволит повысить гибкость и производительность системы, делая её готовой к дальнейшему расширению.

Листинг кода представлен в приложении А.

4 ПРОЕКТИРОВАНИЕ ФУНКЦИЙ И АЛГОРИТМОВ, НЕОБХОДИМЫХ ДЛЯ РАБОТЫ ПРИЛОЖЕНИЯ

4.1 Общий анализ алгоритмов

В настоящее время качественные игры должны быть высокотехнологичными и нересурсозатратными для запуска на различных моделях устройств игроков. Для создания максимально оптимизированной игры необходимо обдуманно подойти к выбору алгоритмов, реализующих выбранное приложение.

В данной курсовой работе были выделены следующие функции:

- 1 Генерация полей различного размера.
- 2 Возможность сыграть в игру.
- 3 Наличие удобных заметок для игры.
- 4 Подсчет времени для текущего игрового поля.
- 5 Проверка возможности решения при текущей раскраске поля.
- 6 Ведение общей и личной статистики.

Для реализации выделенных функций были выделены 2 основных алгоритма:

- 1 Алгоритм решения заданного поля.
- 2 Алгоритм генерации поля заданного размера.

4.2 Алгоритм решения заданного поля

Для генерации корректного поля необходимо уметь решать заданное поле и проверять его на уникальность. Алгоритм, реализованный в классе `HitoriSolver`, сочетает эвристические методы с рекурсивным поиском, чтобы найти решение, соответствующее правилам `Hitori`. Рассмотрим шаги алгоритма, его реализацию и асимптотическую сложность.

Как уже было сказано в главе пункте 3.5 главы 3, алгоритмическая основа модуля включает несколько этапов:

- 1 Инициализация поля.
- 2 Эвристическая обработка.
- 3 Рекурсивный.
- 4 Проверка решения.

Эвристическая обработка включает в себе заполнение поля на основе его основных ходов, рассмотренных в пункте 1.2 главы 1.

Каждая ячейка может иметь три состояния: закрашенная, не закрашенная и

неопределенная. Итеративно применяются проверки для устранения противоречий, пока поле не перестанет изменяться:

1 PaintedProcessing: Для каждой закрашенной ячейки проверяет соседей (сверху, снизу, слева, справа). Если сосед тоже закрашен, возвращается false (нарушение правил). Иначе сосед помечается как незакрашенный.

2 UnPaintedProcessing: Для каждой незакрашенной ячейки проверяет строку и столбец. Если есть дублирующее число, оно должно быть закрашено, иначе возвращается false.

3 DisconnectedProcessing: Использует BFS (CountBFS) для проверки связности незакрашенных ячеек. Если закрашивание ячейки с неопределенным состоянием приводит к разрыву связности, она помечается как незакрашенная.

4 ThreeIdenticalProcessing: Если три одинаковых числа расположены подряд в строке или столбце, средняя ячейка должна быть незакрашенной.

5 TwoIdenticalProcessing: Если два соседних числа одинаковы, остальные их дубликаты в строке или столбце должны быть закрашены.

6 BetweenIdentical: Если ячейка находится между двумя одинаковыми числами в строке или столбце, она должна быть незакрашенной.

Теперь, после эвристической обработки поля могут возникнуть два случая:

1 Клеток с неопределенным состоянием не осталось

2 Остались неопределенные клетки

В 1-м случае мы уже имеем решение заданного поля. При чем мы можем утверждать, что наше поле – уникально, так как для его решения мы не использовали перебор. Однако, чтобы наше поле было поистине уникальным оно не должно иметь еще другие решения, это будет гарантироваться при генерации полей (см. пункт 4.3).

Во 2-м случае для решения головоломки надо перебирать закрашенные клетки. Алгоритм рекурсивно пробует закрасить или оставить незакрашенной каждую неопределенную ячейку, вызывая себя для проверки новых состояний (эвристическую обработку). Первое найденное решение сохраняется в статическом поле GeneralAnswer.

Оценим сложность данного алгоритма. Каждая проверка в эвристической обработке выполняется за $O(n^2)$, но количество итераций зависит от структуры поля. В худшем случае может быть $O(n^2)$ итераций, что в общей сложности дает $O(n^4)$, где n – размер поля Nitori.

В рекурсивном алгоритме, в худшем случае, если после эвристик остаются k неопределенных ячеек, сложность составляет $O(2^k)$, так как для каждой ячейки пробуются два состояния (закрашиваем ее или нет). В реальных задачах k обычно достаточно мало благодаря эвристикам, и достаточно перебрать сильно

малое количество клеток чтобы эффективно заработали эвристические методы.

Итого, в общем случае получаем асимптотику $O(n^4 + 2^k)$ в худшем случае, где k достаточно малое число $\leq n^2$. Обычно размер поля Hitori не превосходит 20, что делает данную асимптотику приемлемой для таких ограничений. Однако может попасться поле, которое данный алгоритм будет решать немалое количество секунд из-за слишком долгого перебора. Но таких полей в нашем проекте не будет (мы специально не будем их генерировать, см. пункт 4.3), так как они имеют меньший интерес у пользователей.

4.3 Алгоритм генерации поля заданного размера

Было исследовано немало статей по поводу генерации уникального поля Hitori и стало ясно, что обычное случайное заполнение полей очень редко приводит к желаемому результату. К примеру, в источнике [5, стр. 32] было сказано, что такой подход почти никогда не приводил к нужному результату. А в источнике [7], содержащего около двух сотен тысяч полей Hitori, находилось всего 5 полей размером 20x20.

Было решено сделать генерацию в отдельном проекте в среде разработки CLion с последующим сохранением полей в файлы. В итоге удалось придумать следующий хитрый алгоритм, состоящий из четырех этапов:

- 1 Генерация возможного ответа (только закрашенные клетки без чисел такие, что белые клетки образуют связную область).

- 2 Заполнение не закрашенных клеток в сгенерированном ответе числами в соответствии с правилами (без повторов в строке/столбце).

- 3 Заполнение закрашенных клеток числами на основе располагающихся в не закрашенных клетках чисел в том же столбце и/или строке.

- 4 Проверка поля на уникальность и сохранение полей в файлы.

1-й этап реализован в цикле while: изначально все клетки имели неопределенное состояние, затем на каждом шаге случайно выбиралась неопределенная клетка и мы пытались ее закрасить. Если ее закрасить не получается, то есть область белых клеток перестанет быть связной (использовался DFS для этой проверки), то клетка помечалась как не закрашенная и цикл повторялся. Если удалось закрасить клетку, то все ее соседи помечались как не закрашенные. Если не удалось составить поле, то пробуем сделать это заново. Нетрудно заметить, что такая генерация удовлетворяет правилам игры. Пусть размер поля n , то есть имеется n^2 клеток, для которых мы запускаем DFS ($O(n^2)$). Итого в худшем случае этот этап работает за $O(n^4)$.

2-й этап был реализован через рекурсию. Для каждой не закрашенной

клетки составлялся вектор из возможных значений чисел, которые могут стоять в данной клетке. Затем этот вектор случайно перемешивался и мы рекурсией пробовали заполнять эту клетку числами из этого вектора сохраняя первый получившийся ответ. Тяжело оценить асимптотику данного этапа, однако амортизировано он работает за $O(n^4)$, так как мы для каждой из n^2 клеток строим вектор из чисел от 1 до n которых нет в строке и столбце ($O(n)$), затем этот вектор перемешиваем и пробуем поставить каждое число в данную клетку (еще $O(n)$ в худшем случае).

3-й этап просто проходится по черным клеткам и случайно заполняет их числами из той же строки/столбца. Причем большая вероятность заполнения тем числом, которое есть и в той же строке, и в том же столбце что и наша клетка. Это позволяет лучше заполнять поле и увеличивает вероятность генерации уникального поля. Работает за $O(n^3)$ – проход по всем $O(n^2)$ клеткам и для каждой из них просматривает столбец и строку (еще $O(n)$).

4-й этап запускает алгоритм решения (пункт 4.3) и проверяет его на корректность и уникальность. Если программа долго не находит решение для поля, то данное поле мы просто не сохраняем и останавливаем программу (поэтому как было сказано в пункте 4.2, полей, нуждающихся в большом переборе, в нашем проекте не будет).

Общая сложность алгоритма около $O(n^4)$, однако при больших n она сильно возрастает из-за большего количества попыток генерации закрашенных клеток и рекурсивного заполнения не закрашенных клеток.

Итого получилось сгенерировать по 20000 полей размером 5×5 , по 10000 полей размеров 10×10 , по 5000 полей размеров 15×15 , 5000 полей размера 20×20 и 2000 уникальных полей размером 20×20 . Уникальные поля размером 20×20 генерировались около 30 минут с несколькими остановками долгого рекурсивного перебора. Я считаю данный результат достаточно хорошим.

ЗАКЛЮЧЕНИЕ

Разработка аналога игры Hitori в рамках данной курсовой работы позволила создать функциональное и интерактивное приложение, реализующее японскую логическую головоломку с использованием кроссплатформенного фреймворка Qt и языка программирования C++. Проект успешно достиг поставленной цели – разработки компьютерной версии игры, обеспечивающей увлекательный игровой процесс и развитие логического мышления пользователей. Реализация модульной архитектуры, включающей модули аутентификации, главного меню, игрового окна, решения головоломки и хранения данных, обеспечила гибкость и масштабируемость системы, что соответствует современным требованиям к программному обеспечению.

Алгоритмы решения и генерации полей, разработанные в проекте, демонстрируют высокую эффективность для заданных размеров (5x5, 10x10, 15x15, 20x20). Алгоритм решения, основанный на эвристической обработке и рекурсивном поиске, обладает асимптотической сложностью $O(n^4 + 2^k)$, где эвристики значительно сокращают количество перебираемых состояний, делая его пригодным для практического применения. Алгоритм генерации полей, включающий случайное определение закрашенных ячеек, рекурсивное заполнение чисел и проверку связности, также работает в пределах $O(n^4)$, успешно создав более 75 000 полей различной сложности, включая 2000 уникальных полей 20x20 за 30 минут. Эти результаты подтверждают надежность подхода.

Приложение удовлетворяет ключевым требованиям, таким как генерация разнообразных полей, учет времени, индикация правильности решения и сохранение статистики, что превосходит многие существующие аналоги, предлагающие ограниченный бесплатный контент.

Проект способствовал развитию навыков программирования, работы с Qt и проектирования архитектуры, что имеет ценность для профессионального роста. В будущем планируется доработка интерфейса, расширение функциональности для повышения привлекательности приложения, что сделает его конкурентоспособным среди существующих аналогов Hitori. Таким образом, работа представляет собой прочный фундамент для создания качественного игрового продукта с потенциалом дальнейшего развития.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

- [1] Доманов, А. Т. Стандарт предприятия / А. Т. Доманов, Н. И. Сорока. – Минск : БГУИР, 2024. – 178 с.
- [2] Hitori rules and background [Электронный ресурс]. – Режим доступа : <https://gclnk.com/pqcX22SL>
- [3] Wikipedia. Хитори [Электронный ресурс]. – Режим доступа : <https://gclnk.com/FnBsH1Vk>
- [4] Хитори: логическая игра онлайн [Электронный ресурс]. – Режим доступа : <https://gclnk.com/bcnixUpp>
- [5] Solving and Generating Puzzles with a Connectivity Constraint / G. van der Knijf [Электронный ресурс]. – Режим доступа : <https://gclnk.com/NhbcM1SV>
- [6] Hitori Game Solver and Generator / Columbia University [Электронный ресурс]. – Режим доступа : <https://gclnk.com/jwNQP8mj>
- [7] Puzzle database [Электронный ресурс]. – Режим доступа : <https://gclnk.com/zPaJGIAP>

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг программного кода

Листинг А.1 – Основной алгоритм генерации поля

```
int main() {
    srand(time(0));
    string File = to_string(n) + "x" + to_string(n) + ".txt";
    //string File_unique = to_string(n) + "x" + to_string(n) + "_unique.txt";
    string FileCnt = to_string(n) + "x" + to_string(n) + "_cnt.txt";
    //string FileUniqueCnt = to_string(n) + "x" + to_string(n) + "_unique_cnt.txt";

    ifstream file(FileCnt);
    file >> GlobalHitoriCount;
    file.close();

    while(1) {
        answerFind = false;
        vector<pair<int, int>> pos;
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++) {
                answer[i][j] = 2;
                field[i][j] = 0;
                answerField[i][j] = 100;
                pos.push_back({i, j});
            }

        while(pos.size() != 0) {
            int curPos = rand() % pos.size();
            int x, y;
            tie(x, y) = pos[curPos];

            answer[x][y] = 1;
            pos.erase(pos.begin() + curPos);

            if (!good()) {
                answer[x][y] = 0;
                continue;
            }

            if (x > 0) {
                auto nPos = find(pos.begin(), pos.end(), make_pair(x - 1, y));
                if (nPos != pos.end()) {
```

```

        pos.erase(nPos);
        answer[x - 1][y] = 0;
    }
}

if (y > 0) {
    auto nPos = find(pos.begin(), pos.end(), make_pair(x, y - 1));
    if (nPos != pos.end()) {
        pos.erase(nPos);
        answer[x][y - 1] = 0;
    }
}

if (x < n - 1) {
    auto nPos = find(pos.begin(), pos.end(), make_pair(x + 1, y));
    if (nPos != pos.end()) {
        pos.erase(nPos);
        answer[x + 1][y] = 0;
    }
}

if (y < n - 1) {
    auto nPos = find(pos.begin(), pos.end(), make_pair(x, y + 1));
    if (nPos != pos.end()) {
        pos.erase(nPos);
        answer[x][y + 1] = 0;
    }
}
}

rec(0, 0);

if (!answerFind)
    continue;

bool bad = false;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        if (!(answerField[i][j] >= 1 && answerField[i][j] <= n) && !answer[i][j])
            bad = true;
    }
}
if (bad)
    continue;

```

```

fillBlack();

bad = false;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        if (!(answerField[i][j] >= 1 && answerField[i][j] <= n))
            bad = true;
    }
}
if (bad)
    continue;

//if (!IsUnique())
//    continue;

SaveToFiles(File, FileCnt);

if (GlobalHitoriCount == 5000)
    break;
}
return 0;
}

```

Листинг А.2 – Рекурсивная функция заполнения не закрашенных клеток для генерации поля

```

void rec(int i, int j) {
    if (answerFind)
        return;
    if (answer[i][j]) {
        if (j == n - 1 && i == n - 1) {
            if (!answerFind)
                saveField();
            answerFind = true;
            return;
        }
        if (j == n - 1)
            rec(i + 1, 0);

        if (answerFind)
            return;
        rec(i, j + 1);
    }
    if (answerFind)

```

```

    return;

    for (int k = 0; k <= n; k++)
        was[k] = 0;
    for (int k = 0; k < n; k++) {
        was[field[i][k]] = was[field[k][j]] = true;
    }

    vector<int> ost;
    for (int i = 1; i <= n; i++) {
        if (!was[i])
            ost.push_back(i);
    }

    if (ost.size() == 0)
        return;

    std::random_device rd;
    std::mt19937 g(rd());
    shuffle(ost.begin(), ost.end(), g);

    for (auto u : ost) {
        field[i][j] = u;
        if (i == n - 1 && j == n - 1) {
            if (!answerFind)
                saveField();
            answerFind = true;
            return;
        }
        if (j == n - 1)
            rec(i + 1, 0);
        else
            rec(i, j + 1);
        if (answerFind)
            return;
    }
}

```

Листинг А.3 – Функции заполнения закрашенных клеток для генерации

```

void fillBlack() {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (!answer[i][j])

```

```

        continue;

        vector<int> curNumbers;
        for (int k = 0; k < n; k++) {
            if (!answer[i][k])
                curNumbers.push_back(field[i][k]);
            if (!answer[k][j])
                curNumbers.push_back(field[k][j]);
        }

        std::random_device rd;
        std::mt19937 g(rd());
        shuffle(curNumbers.begin(), curNumbers.end(), g);

        int pos = rand() % curNumbers.size();
        answerField[i][j] = curNumbers[pos];
    }
}

```